MIPS Processor: CPU Datapath and Instructions

---

1 Introduction to CPU Performance Factors

- **CPU performance** depends primarily on three factors:
  1. **Instruction count** — number of instructions executed in a program.
  2. **Cycles per instruction (CPI)** — average number of clock cycles each instruction takes.
  3. **Cycle time** — duration of one clock cycle determined by hardware.
- **Instruction count** is influenced by the **Instruction Set Architecture (ISA)** and compiler optimizations.
- **CPI** and **cycle time** are primarily determined by the CPU hardware design.
- In this guide, we focus on **two MIPS implementations**:
  - A **simple** single-cycle design.
  - A **pipelined** and more realistic design.
- We will focus on a simple subset of instructions showcasing key aspects:
  - Arithmetic/Logical: `add`, `addi`
  - Memory reference: `lw`, `sw`
  - Control transfer: `beq`, `j`

2 Building the Datapath

CPU Datapath Components

A **datapath** is a collection of hardware elements that perform data processing and address calculations in the CPU. For MIPS, the key components are:

- **PC Register**: Keeps track of the address of the current instruction.
- **Instruction Memory**: Stores the program instructions.
- **Register File**: Contains CPU registers—read operands and write results.
- **ALU (Arithmetic Logic Unit)**: Performs arithmetic and logic operations, computes memory addresses and branch targets.
- **Data Memory**: Used for loading and storing data values.

3 Instruction Fetch Cycle

- The **Program Counter (PC)** is a special register that holds the memory address of the instruction the CPU will execute next.
- During the **fetch cycle**, the CPU reads (fetches) the instruction stored at the address contained in the PC from the instruction memory.
- Since each MIPS instruction is **4 bytes (32 bits)** and instructions are **word-aligned**, the PC is incremented by 4 after each fetch to point to the next sequential instruction.
- This updated PC is stored back into the PC register and will be used during the next cycle.
- If control instructions like branches or jumps occur, they may override the default PC + 4 update to redirect program flow.

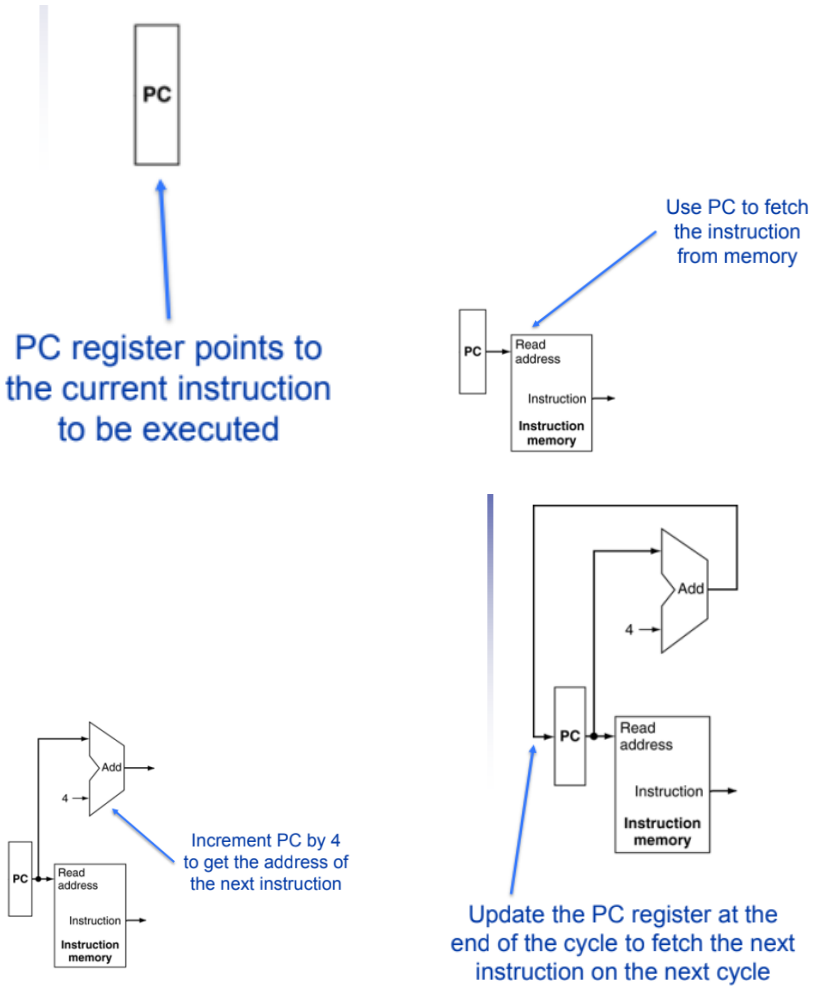


Figure 1: Instruction Fetch Cycle in a Processor: Visualizing PC operations and memory access.

Cycle Steps for Instruction Fetch

1. **Access Instruction Memory:** Use the address in the PC to locate the instruction in instruction memory.
2. **Read Instruction:** Fetch the 32-bit instruction (4 bytes) stored at that address.
3. **Prepare for Next Instruction:** Add 4 to the current PC to get the address of the next instruction (sequential execution).
4. **Update the PC Register:** Store the new PC value (PC + 4) back in the PC register for the next cycle.

4 R-type Instructions (Register-to-Register Operations)

Example: `add $t1, $a0, $s2`

```
add $t1, $a0, $s2                    # $t1 = $a0 + $s2
```

- R-type instructions perform arithmetic or logical operations between two registers.
- The general format: `opcode | rs | rt | rd | shamt | funct`
  - `rs` and `rt` are the source registers.
  - `rd` is the destination register.
  - `shamt` is the shift amount (used only for shift instructions).
  - `funct` specifies the exact ALU operation (e.g., 'add', 'sub', etc.)
- In this example:
  - `rs = $a0`, `rt = $s2`, `rd = $t1`
  - `opcode = 000000` (for all R-type instructions)
  - `funct = 100000` (for `add`)

CPU Datapath Steps

1. The instruction is fetched from memory and decoded by the control unit.
2. The values of registers $a0 and $s2 are read from the register file.
3. These 32-bit values are sent to the ALU.
4. The ALU performs a 32-bit addition (4 bytes = 32 bits) and produces a result.
5. The result is written back to the destination register $t1.

Control Signals in Action

- **RegWrite = 1:** Enables writing the ALU result into the register file. Without this, no update occurs in $t1.
- **ALUOp:** A 2-bit control field that the control unit uses to describe what type of ALU operation should be performed:
  - For R-type: ALUOp = 10 → The ALU control unit checks the `funct` field to decide the exact operation.

- **ALU Control:** Decoded from ALUOp + funct. For example:
  - ALUOp = 10, funct = 100000 → ALUControl = 0010 (which means addition)

Why 32-bit Operations?

- MIPS is a 32-bit architecture: each register is 32 bits wide.
- Therefore, arithmetic operations operate on 4-byte (32-bit) values.
- ALU must support 32-bit input buses to handle the data width of operands.

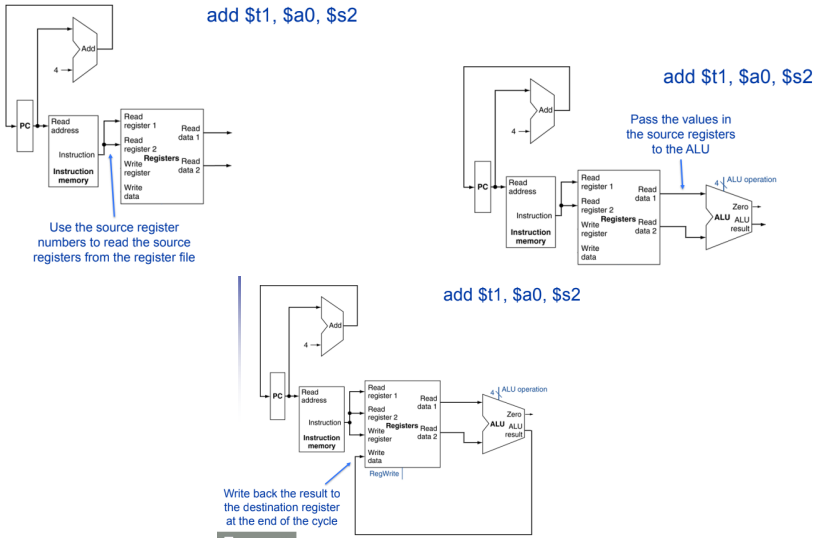


Figure 2: Datapath illustrating the flow of an R-type instruction like `add $t1, $a0, $s2`.

5 Immediate Instructions (I-Type)

Example: `addi $t0, $s0, 4`

```
addi $t0, $s0, 4                    # $t0 = $s0 + 4
```

- I-type (Immediate-type) instructions use:
  - One register operand (read from register file).
  - One 16-bit immediate constant encoded in the instruction.
  - A destination register for storing the result.

- This format is used for operations like *addi*, *andi*, *ori*, *lw*, *sw*, *beq*, and others.
- **Instruction format:** `[opcode (6)] [rs (5)] [rt (5)] [immediate (16)]`
  - `rs`: Source register
  - `rt`: Destination register (for most ALU immediates)
  - `immediate`: Constant value (signed)

Datapath and Control Steps

1. Fetch the instruction from memory.
2. Decode the instruction and read the value from register `$s0`.
3. **Sign-extend** the 16-bit immediate value to 32 bits. This is required to match the 32-bit ALU input size.
4. The control signal **ALUSrc** = 1 is set, which tells the **MUX** to choose the sign-extended immediate (instead of a second register).
5. The ALU performs the operation (e.g., addition).
6. The result is written to destination register `$t0` (if RegWrite = 1).

What Do ALUSrc, MUX, and SignExtend Do?

- **SignExtend:** Extends a 16-bit signed number to 32 bits by replicating the sign bit (bit 15) into the upper 16 bits. Example:

  `0000_0000_0000_1111` ⇒ `0000_0000_0000_0000_0000_0000_0000_1111`

  `1000_0000_0000_1111` ⇒ `1111_1111_1111_1111_1000_0000_0000_1111`

- **ALUSrc:** A control signal that selects the second input to the ALU:
  - If ALUSrc = 0 : the second ALU input comes from a register (R-type).
  - If ALUSrc = 1 : the second ALU input comes from the sign-extended immediate (I-type).
- **MUX (Multiplexer):** A 2-input selector that uses ALUSrc as a control line. It chooses between:
  1. `Read data 2` (register content).
  2. `Sign-extended immediate`.

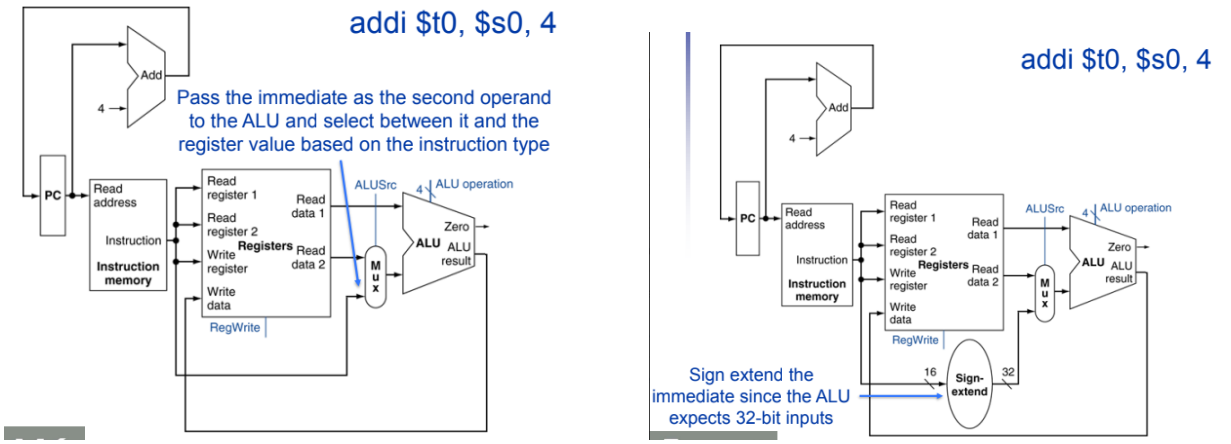


Figure 3: Datapath for Immediate Instructions. Shows how ALUSrc, SignExtend, and MUX interact.

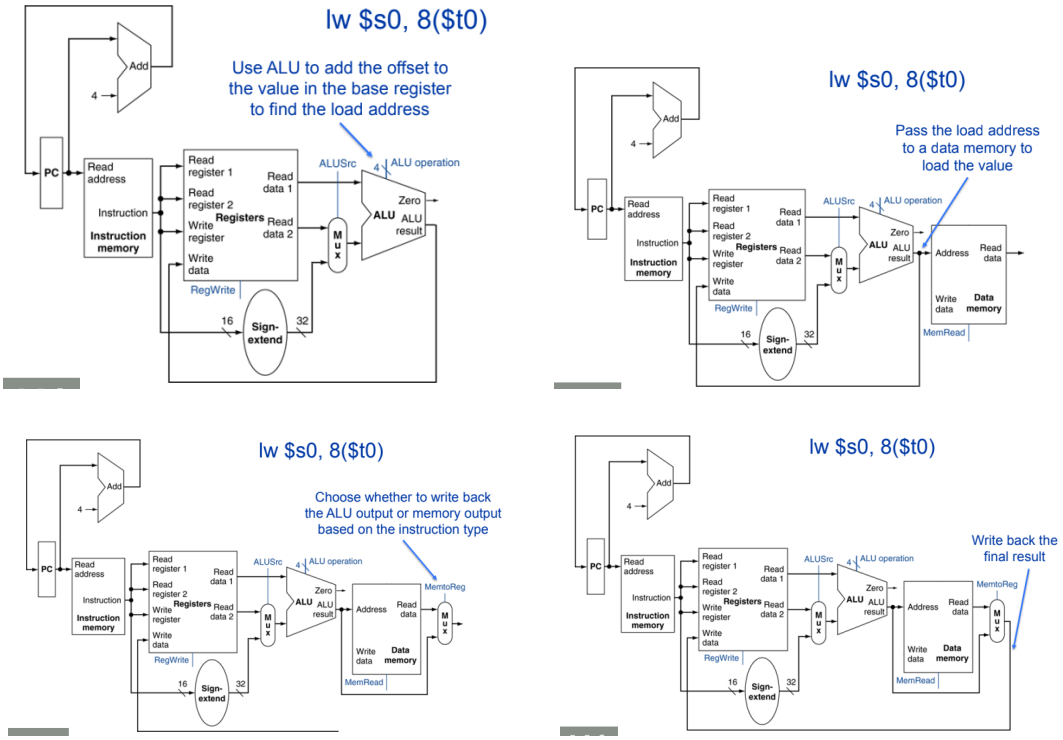
6 Load Instructions (Memory to Register)

Example: `lw $s0, 8($t0)`

lw $s0, 8($t0)

- The instruction `lw` (load word) is used to read a 32-bit value from memory and store it into a register.
- It uses a base register and a 16-bit offset to calculate the final memory address.
- **Address Calculation:**
  - The base register ($t0) holds a memory address.
  - The 16-bit immediate offset (8) is sign-extended to 32 bits.
  - The ALU adds the base register and the extended offset to compute the effective memory address.
- **Execution Steps:**
  1. Read the base register ($t0) from the register file.
  2. Sign-extend the offset and compute the effective address using the ALU.
  3. Activate the control signal **MemRead** to read data from data memory at that address.
  4. Data memory sends the 32-bit word stored at the computed address.
  5. A multiplexer selects between the ALU result and memory output. For load, the memory output is selected since **MemtoReg = 1**.
  6. The loaded value is written to the destination register ($s0) using the **RegWrite** control signal.

- **Key Control Signals:**
  - **MemRead = 1**: Enables reading from data memory.
  - **MemtoReg = 1**: Selects memory output (not ALU output) for writing to register.
  - **ALUSrc = 1**: Selects immediate offset as ALU input instead of a second register.
  - **RegWrite = 1**: Enables writing into the destination register.
- **What is Data Memory?**
  - Data memory is a memory segment used to store and retrieve data (not instructions).
  - It is accessed by memory-related instructions like lw, sw, etc., using effective addresses generated by the ALU.

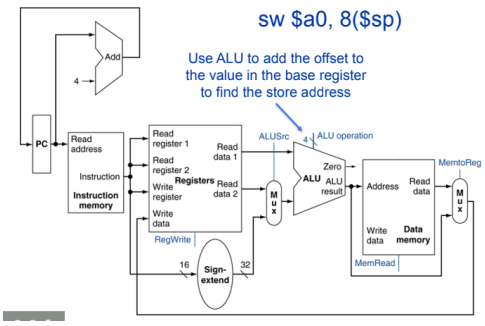


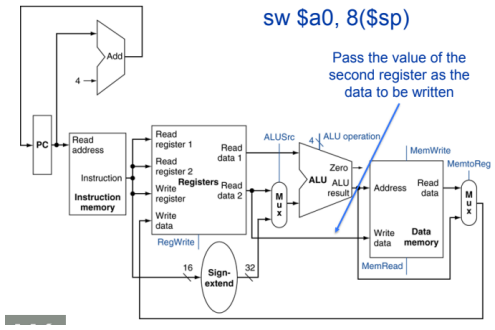
7 Store Instructions (Register to Memory)

Example: `sw $a0, 8($sp)`

sw $a0, 8($sp)

- The instruction `sw` (store word) stores a 32-bit value from a register into memory.
- It also uses a base address register and an immediate offset to compute the final address.
- **Execution Steps:**
  1. Read the base register ($sp) from the register file.
  2. Sign-extend the 16-bit offset (8) to 32 bits.
  3. Use the ALU to compute the effective memory address by adding the base and offset.
  4. Read the data to be stored from the source register ($a0).
  5. Activate the **MemWrite** control signal to write the value into data memory at the computed address.
- **Key Control Signals:**
  - **MemWrite = 1**: Enables writing to data memory.
  - **RegWrite = 0**: No register is written (since we are storing to memory).
  - **ALUSrc = 1**: Immediate value (offset) is used as input to the ALU.
- **Additional Notes:**
  - The ALU is still used to compute the target address, just like in lw.
  - Data memory performs the write operation when **MemWrite** is enabled.
  - No **MemtoReg** control is involved, because data is not going into a register.





8 Branch Instructions

Example: `beq $t0, $s0, offset`

```
beq $t0, $s0, offset
```

- Branch instructions like `beq` (branch if equal) are used to alter the control flow of the program based on a condition.
- These instructions compare the values in two registers and, if the condition is met, cause the program to branch to a new instruction address.

Execution Steps

1. Read the source registers $t0 and $s0 from the register file.
2. The ALU subtracts the values of the two registers:

$$\text{ALUResult} = \$t0 - \$s0$$

   If the result is zero, this indicates that the two registers are equal.
3. The ALU sets the Zero flag if the result of subtraction is 0. This Zero flag is used by the control unit to decide whether to take the branch.
4. The 16-bit offset from the instruction is sign-extended to 32 bits.
5. The offset is then shifted left by 2 bits. This is because MIPS instructions are word-aligned (each instruction is 4 bytes), so we multiply the offset by 4 (shift left by 2) to get the correct byte address:

$$\text{BranchOffset} = \text{SignExt}(\text{offset}) \ll 2$$

6. The branch target address is calculated as:

$$\text{BranchTarget} = PC + 4 + \text{BranchOffset}$$

   where PC is the address of the current instruction.
7. A multiplexer (MUX) is used to select between:
   - PC + 4 (the address of the next sequential instruction), and
   - Branch target address (if the branch is taken).

   The selection is controlled by the signal **PCSrc**.
8. If the branch condition is met (Zero = 1) and the Branch control signal is set to 1, the PCSrc signal becomes 1 and the branch address is selected as the next value of the Program Counter (PC). Otherwise, the next PC is just PC + 4.

Control Signals

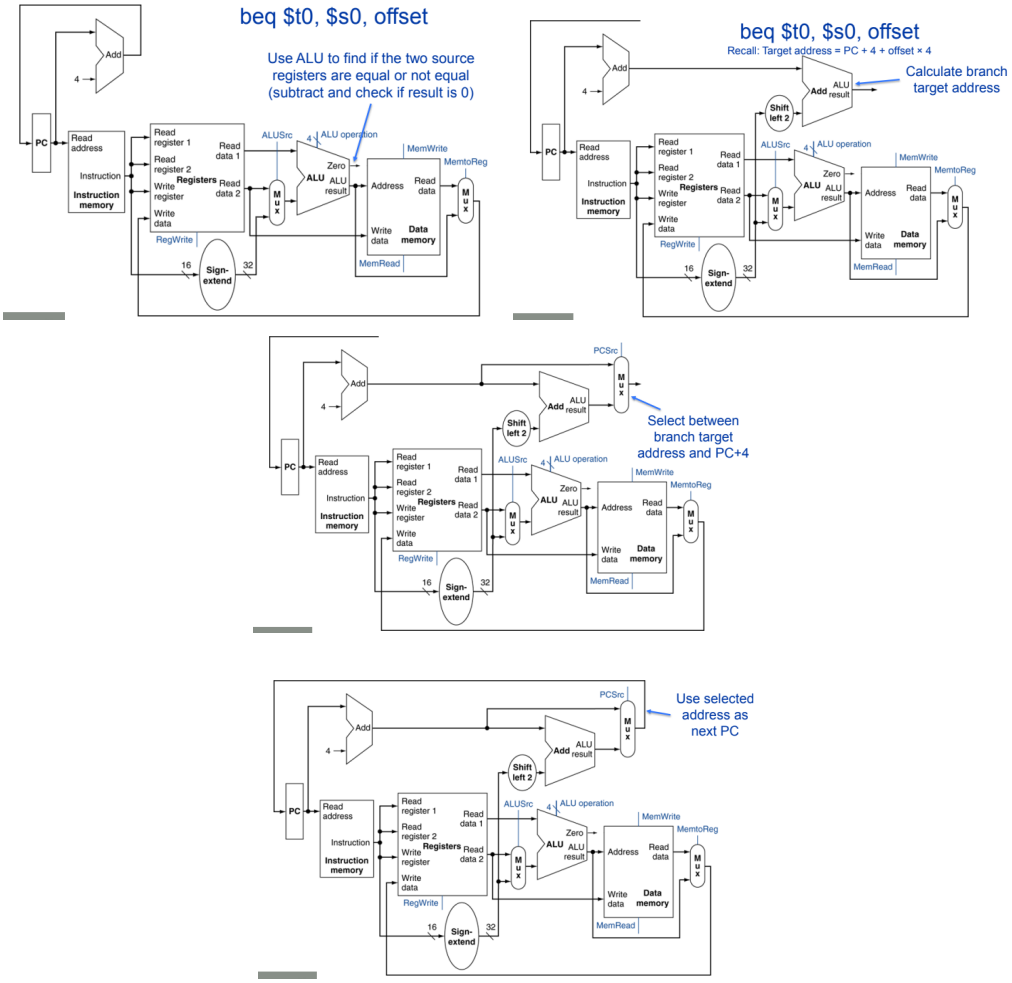
- **Branch = 1**: This signal indicates a branch instruction (like beq) is being executed.
- **Zero**: Comes from ALU and indicates whether the two registers are equal (Zero = 1) or not.
- **PCSrc = Branch AND Zero**: A logical AND of the control signal and ALU's Zero output. This determines whether the branch is taken.

Why Shift Left by 2?

- MIPS uses byte addressing, and each instruction is 4 bytes.
- The offset in the instruction is in words, not bytes. Shifting left by 2 bits converts word offset to byte offset.

Branch Target Selection Using MUX

- A 2-to-1 multiplexer chooses between PC + 4 and the branch target.
- Controlled by PCSrc.
- Output of the multiplexer becomes the new PC.



Summary: Instruction Data Flow

General Instruction Cycle Summary

1. **Fetch**: Get instruction from instruction memory using PC.
2. **Decode/Register Fetch**: Extract fields, read source registers.
3. **Execute/Address Calculation**: Use ALU to perform arithmetic, logical, or address calculations.
4. **Memory Access**: Read or write data memory for loads/stores.
5. **Write Back**: Write results to register file if needed.
6. **Update PC**: Increment PC or set branch/jump target for next instruction.

This fundamental datapath concept supports all MIPS instructions and can be extended for pipelining and other optimizations.